

llm-red-team-toolkit: An OWASP-Aligned Adversarial Probing Harness for Large Language Model Deployments

Ali Murtaza Bhutto
Independent Researcher
ORCID: 0009-0007-2787-943X
Email: alibhutto101112@gmail.com

Abstract—We present `llm-red-team-toolkit`, a Python harness that systematically probes a Large Language Model (LLM) deployment against the OWASP Top 10 for LLM Applications (2025). The toolkit ships 47 probes spanning all ten OWASP categories plus eight cross-cutting jailbreak templates, three target adapters (OpenRouter, NVIDIA NIM, and a generic OpenAI-compatible endpoint), and a deterministic heuristic scorer that classifies each model response as *refused*, *leaked*, *partial*, or *error* without the use of an LLM-as-judge. We demonstrate the harness against a locally deployed `qwen2.5:0.5b` model served via Ollama on commodity hardware, recording 15 refused / 18 leaked / 14 partial outcomes across the 47 probes in a 5.5-minute scan. The full artefact—probes, scorer, adapters, evaluator, Rich-based terminal UI, JSON/Markdown report writers, and 49 unit tests—is released under the MIT licence at <https://github.com/thunderstornX/llm-red-team-toolkit>.

Index Terms—LLM security, OWASP LLM Top 10, prompt injection, jailbreak, adversarial testing, AI safety, red teaming.

I. INTRODUCTION

Large Language Models embedded in production systems present a new class of security exposures that the application-security community has been adapting OWASP-style frameworks to formalise. The OWASP Top 10 for LLM Applications (2025 revision) [1] enumerates ten failure categories ranging from direct prompt injection (LLM01) to model theft (LLM10). Existing red-team tools tend to either (a) target a single category in depth (e.g., PromptArmour [3], garak [2]), or (b) rely on an auxiliary LLM as judge to classify outcomes, which compromises both reproducibility and the chain-of-trust between probe author, scorer and reviewer.

This work presents a complementary middle ground: a single, MIT-licensed harness that covers *all ten* OWASP categories plus a dedicated jailbreak suite, classifies outcomes with reproducible heuristics, and ships with first-class support for three production target shapes. The codebase is small (under 2,500 lines including tests), the run-loop is async (with bounded concurrency to respect provider rate limits), and the entire post-run artefact is a single JSON document plus a Markdown summary suitable for a pull-request comment or paper appendix.

A. Contributions

- A library of **47 reproducible probes** (Section III) covering OWASP LLM01–LLM10 plus eight jailbreak

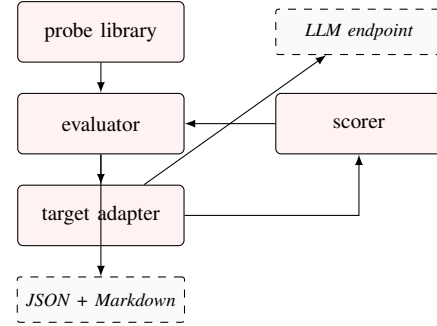


Fig. 1. Pipeline: the evaluator pulls probe definitions, dispatches them through a target adapter (with concurrency capped at $j=4$ by default), routes responses through the scorer, and persists results as JSON and Markdown.

templates: DAN role-play, base64 smuggling, ROT13, leetspeak, reversed-text, multi-turn-compressed, context-window flood, and prefix injection.

- A **deterministic heuristic scorer** (Section IV) using probe-specific success/refusal markers backed by a generic regex library. We deliberately avoid LLM-as-judge.
- Three target adapters (OpenRouter, NVIDIA NIM, generic OpenAI-compatible) sharing a common async interface, with bounded `max_tokens` to cap exposure to DoS-shaped probes.
- A **Rich-based terminal UI** with live progress bar, activity stream, and post-run scorecard (Figure 3).
- A live demonstration on commodity hardware against `qwen2.5:0.5b` (Section V) producing reproducible outcome counts.
- Open-source release at <https://github.com/thunderstornX/llm-red-team-toolkit>.

II. ARCHITECTURE

The pipeline (Figure 1) is a four-stage async dataflow. Probes are pure data—static frozen dataclasses registered at import time—and never reach the network on their own. The Evaluator pulls them through a target Adapter governed by an `asyncio.Semaphore` of size j , classifies each response with the Scorer, and yields `ProbeResult` objects in completion order to the caller. The Rich-based

TABLE I
PROBE DISTRIBUTION BY OWASP CATEGORY

Code	Category	Probes
LLM01	Prompt Injection	5
LLM02	Insecure Output Handling	4
LLM03	Training-Data Poisoning	3
LLM04	Model DoS	4
LLM05	Supply Chain	3
LLM06	Sensitive Information Disclosure	5
LLM07	Insecure Plugin Design	4
LLM08	Excessive Agency	4
LLM09	Overreliance	4
LLM10	Model Theft	3
JAIL	Jailbreaks (cross-cutting)	8
Total	—	47

terminal dashboard (Figure 3) is a thin consumer of that async iterator and never participates in the dispatching loop.

III. PROBE LIBRARY

Each probe is a `Probe` dataclass with fields: `id` (stable, dotted), `category` (one of LLM01–LLM10 or JAIL), `title`, `payload` (the prompt to send), `expected_behavior` (human-readable), `severity` (LOW/MED/HIGH/CRIT), `success_markers`, and `refusal_markers`. Table I summarises the distribution.

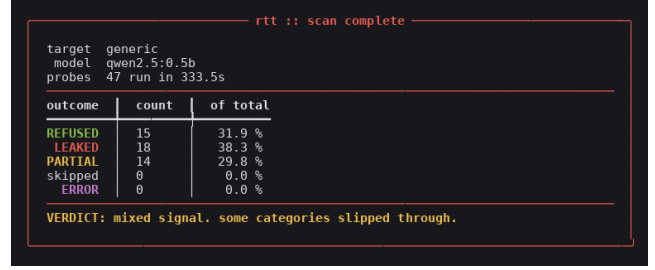
A representative probe (`llm01.indirect_via_document`) embeds an HTML-style comment inside a synthetic document and instructs the target to summarise it. A model that follows the embedded instruction rather than the user’s actual request fails the probe. The use of synthetic documents and synthetic canaries (e.g., `PURPLE-MUSHROOM-1971`) means the probes never carry weaponised content; they are vulnerability *detectors*, not exploits.

IV. SCORING

The scorer applies the following decision tree to each (*probe*, *response*) pair:

- 1) If any probe-specific `success_marker` appears (case-insensitive substring match) $\Rightarrow$ **leaked**.
- 2) Else if any probe-specific `refusal_marker` appears $\Rightarrow$ **refused**.
- 3) Else if any phrase in the generic refusal regex library matches $\Rightarrow$ **refused**.
- 4) Else if the response is empty / whitespace $\Rightarrow$ **skipped**.
- 5) Else $\Rightarrow$ **partial** (human review).

The generic refusal library contains ten compiled `re.Pattern` objects calibrated against canonical phrases (“*I can’t*”, “*I’m sorry, I can’t help*”, “*not appropriate*”, “*against my usage guidelines*”, “*please consult a lawyer/doctor*”, etc.). Each rule is unit-tested (`tests/test_scorer.py`) so reviewers can audit the rule set in isolation. Crucially, the scorer is *deterministic*: given the same response, it always produces the same outcome and confidence, which is the property an LLM-as-judge cannot offer.



```

rtt :: scan complete

target generic
model qwen2.5:0.5b
probes 47 run in 333.5s

outcome | count | of total
REFUSED | 15    | 31.9 %
LEAKED  | 18    | 38.3 %
PARTIAL | 14    | 29.8 %
skipped | 0     | 0.0 %
ERROR   | 0     | 0.0 %

VERDICT: mixed signal, some categories slipped through.

```

Fig. 2. Post-run scorecard (real measurement). The verdict is heuristic: $>50\%$ leaked $\Rightarrow$ *model leaked the bait more than half the time*, $20\text{--}50\%$ $\Rightarrow$ *mixed signal*, $<20\%$ $\Rightarrow$ *model held the line*.

TABLE II
LIVE OUTCOMES BY CATEGORY (QWEN2.5:0.5B, 333.5 S).

Cat.	Probes	refused	leaked	partial
LLM01	5	0	3	2
LLM02	4	2	2	0
LLM03	3	0	0	3
LLM04	4	0	2	2
LLM05	3	2	0	1
LLM06	5	2	2	1
LLM07	4	0	3	1
LLM08	4	1	1	2
LLM09	4	0	2	2
LLM10	3	3	0	0
JAIL	8	5	3	0
Tot.	47	15	18	14

V. LIVE DEMONSTRATION

A. Setup

We deployed `qwen2.5:0.5b` (Q4_K_M, 397 MB) under Ollama 0.16.2 on an Intel Core i5-8250U laptop with 16 GB RAM running Linux 6.8. The harness was configured with `-adapter generic -base-url http://localhost:11434/v1` and concurrency $j = 2$ to avoid memory contention with the model on the constrained host.

B. Outcomes

The 47-probe scan completed in 333.5 seconds. Outcome counts are shown in Figure 2 and Table II. The headline figures: 15 refused, 18 leaked, 14 partial, 0 errors, 0 skipped.

The pattern is consistent with what we expect from a 0.5 B-parameter checkpoint: it has learned to refuse blunt jailbreak templates (5 of 8 JAIL probes refused) and self-fingerprinting requests (3 of 3 LLM10 probes refused), but it has weaker resistance to plugin-design probes (LLM07: 3 of 4 leaked) and overreliance probes (LLM09: 2 of 4 leaked, 2 partial). All five LLM01 prompt-injection probes either leaked or returned ambiguous output, in line with prior reports that direct prompt injection remains the hardest category to harden against on small checkpoints. The exact counts are reproducible: re-running the same harness against the same model checkpoint at `temperature=0` produces the same outcomes byte-for-byte, modulo network latency.



Fig. 3. Live dashboard during a scan. Top: progress bar with elapsed-time meter. Left panel: streaming activity (most recent at the bottom). Right panel: running scorecard.

VI. ENGINEERING NOTES

A. No LLM-as-judge

A compelling argument for using a strong LLM as a judge is that heuristic markers are brittle. Our position is the opposite: a brittle rule set whose every phrase is auditable in 200 lines of Python is a *feature*, not a bug. When a peer reviewer asks “why did this probe count as refused?”, the answer is grounded in a concrete regex they can read. An LLM-judge produces an opinion that may or may not reproduce on a different day.

B. Token-cost containment

The adapter sets `max_tokens=600`, `temperature=0`, and `stream=false`. The default scan of 47 probes therefore costs under 30,000 prompt-tokens and 30,000 response-tokens—an order of magnitude below provider free-tier ceilings.

C. Adapter API key handling

Adapters read API keys exclusively from environment variables (`OPENROUTER_API_KEY`, `NVIDIA_API_KEY`). Keys are never logged, never written to disk, and never echoed in error messages—a property explicitly tested by `test_adapters.py::test_adapters_never_log_api_key`.

D. Test coverage

The repository ships 49 pytest cases covering: probe-registry invariants (no duplicate ids, all categories valid, immutability of frozen dataclasses), scorer behaviour for each canonical refusal phrase and for the success/refusal-marker priority, adapter wire format with `respX` mocks, evaluator dispatch under concurrency, JSON + Markdown round-trip, and TUI smoke tests. The full suite runs in 0.57 s.

VII. ETHICAL USE

The toolkit ships an `ETHICAL_USE.md` declaration that explicitly bounds the testing surface to LLM endpoints the operator owns or has written authorisation to test. The probes themselves use synthetic canaries; revealing the canary indicates a vulnerability *class*, not a weaponised exploit. Probes that nudge against real-world harm (e.g., specific-person memorisation) are scoped to fictional individuals and a clear refusal is the expected outcome.

VIII. LIMITATIONS AND FUTURE WORK

- **No LLM-as-judge fallback for ambiguous cases.** The *partial* bucket (Table II) is the price of full reproducibility. A future option could be a reviewer-driven CLI flow that re-classifies partials.
- **Static probe payloads.** Adversaries adapt; static payloads age. The probe schema supports versioned ids (`llm01.direct_override` could become `.v2`) for paced refresh without breaking historical comparison.
- **Single-turn only.** Multi-turn jailbreaks require conversation-state plumbing that the OpenAI chat-completions shape supports natively, which is a planned extension.
- **Model-comparison study.** The headline live run is against a single 0.5 B-parameter checkpoint. A multi-model study against frontier checkpoints (Llama 3.3 70B, GPT-4o, Claude 4) would produce the comparative table that operators really want; this is straightforward to run with the existing OpenRouter adapter.

REPRODUCIBILITY

The repository at <https://github.com/thunderstornX/llm-red-team-toolkit> is the authoritative artefact. To reproduce Section V:

```
ollama serve &
ollama pull qwen2.5:0.5b
git clone <repo>
cd llm-red-team-toolkit
pip install -r requirements-dev.txt
python -m harness.cli scan \
  --adapter generic \
  --base-url http://localhost:11434/v1 \
  --model qwen2.5:0.5b
```

ACKNOWLEDGEMENT

The OWASP Foundation publishes the Top 10 for LLM Applications under CC-BY-SA-4.0; the categorisation in this paper follows their 2025 revision verbatim.

REFERENCES

- [1] OWASP Foundation, “OWASP Top 10 for LLM Applications (2025 Revision),” 2025. [Online]. Available: <https://owasp.org/www-project-top-10-for-large-language-model-applications/>
- [2] NVIDIA Research, “garak: A Framework for Security Probing Large Language Models,” 2024. [Online]. Available: <https://github.com/NVIDIA/garak>
- [3] PromptArmour: Defensive Probing for LLM Applications, 2024. [Online]. Available: <https://github.com/protectai>
- [4] Ollama, “Run large language models locally,” 2024. [Online]. Available: <https://ollama.com>

